

Biomedical Scientific Intelligence System — Product & Technical Specification

1. Overview

This project is a local-first biomedical literature intelligence system built as a Streamlit web app for answering scientific research questions with evidence from the biomedical literature. Instead of only returning a list of papers, it attempts to emulate a research analyst workflow:

- search the biomedical literature,
- retrieve relevant full-text sources,
- parse and structure the information,
- extract atomic scientific claims,
- index the evidence in vector and graph form,
- compare findings across studies,
- detect agreement and contradiction,
- synthesize a final evidence-grounded answer.

The app is designed for biomedical questions such as treatment effectiveness, preventive interventions, risk factors, disease outcomes, and evidence comparison across studies.

Related implementation files:

- [app.py](#) ([./app.py](#))
 - [src/acquisition/europe_pmc.py](#) ([./src/acquisition/europe_pmc.py](#))
 - [src/processing/xml_parser.py](#) ([./src/processing/xml_parser.py](#))
 - [src/extraction/claim_extractor.py](#) ([./src/extraction/claim_extractor.py](#))
 - [src/knowledge/vector_store.py](#) ([./src/knowledge/vector_store.py](#))
 - [src/knowledge/graph_store.py](#) ([./src/knowledge/graph_store.py](#))
 - [src/retrieval/rag_retriever.py](#) ([./src/retrieval/rag_retriever.py](#))
 - [src/analysis/contradiction.py](#) ([./src/analysis/contradiction.py](#))
 - [src/synthesis/answer_generator.py](#) ([./src/synthesis/answer_generator.py](#))
-

2. Why this is special

This system is special because it combines multiple research-analysis layers into one workflow rather than acting as a plain search box.

2.1 It is evidence-first, not just AI chat

Most simple AI tools answer using general knowledge or a single retrieved snippet. This app tries to ground the answer in multiple biomedical publications and makes the reasoning traceable.

2.2 It does not rely on a single retrieval mechanism

The app uses:

- semantic vector retrieval,
- paper-level and claim-level embedding search,
- graph-based related-paper expansion,
- cross-encoder reranking.

This is more robust than a single keyword or single vector search.

2.3 It identifies literature disagreement, not just agreement

A major gap in many research tools is that they summarize the literature without highlighting where evidence conflicts. This app includes a contradiction analysis layer using NLI-style pairwise classification to detect supporting vs conflicting claim pairs.

2.4 It is local-first and privacy aware

The app can run locally with Ollama and does not require sending full paper content to an external commercial service. That matters for academic and biomedical workflows where privacy, reproducibility, and institutional constraints matter.

2.5 It is designed for biomedical reasoning workflows

The system extracts structured scientific details such as:

- population,
- intervention,
- comparator,
- outcome,
- methodology,
- evidence level,
- atomic claims.

This gives it a literature-review style structure rather than generic summarization.

3. Core features and functionality

3.1 Literature search from Europe PMC

The app queries the Europe PMC API for open-access biomedical literature matching the user question. It filters to open-access papers and collects metadata such as:

- PMCID / PMID
- title
- abstract
- authors
- journal
- year
- DOI
- publication type

This provides a live evidence base instead of relying only on static local data.

3.2 Full-text XML retrieval and parsing

When a paper has downloadable open-access XML, the app retrieves and parses the full-text JATS XML using BeautifulSoup and lxml.

It extracts:

- title
- abstract
- section labels such as introduction, methods, results, discussion, conclusion
- paragraphs and key sections
- figure captions and table captions
- keywords and MeSH-like terms

This step improves the quality of downstream extraction compared to relying only on abstracts.

3.3 Structured scientific extraction

The extraction layer turns raw paper text into a structured biomedical record.

It attempts to infer:

- research question
- population studied
- intervention
- comparator

- outcome
- study design and evidence level
- sample size
- follow-up duration
- key findings
- limitations
- atomic scientific claims

The app supports two modes:

- AI mode using an Ollama-hosted Qwen3 model for deeper extraction
- heuristic mode for lightweight local analysis when LLM access is unavailable

3.4 ChromaDB vector indexing

The app indexes:

- paper chunks from abstracts/full text,
- atomic claims separately.

This enables semantic retrieval over both complete literature chunks and extracted finding-level statements.

3.5 NetworkX knowledge graph

The app builds a knowledge graph with nodes such as:

- paper nodes,
- claim nodes,
- author nodes,
- concept nodes,
- relationship edges such as AUTHORED_BY, MENTIONS, HAS_CLAIM, SUPPORTS, CONTRADICTS.

This gives the system a second form of retrieval beyond similarity search and supports graph expansion from a seed paper to related work.

3.6 Hybrid retrieval

The retrieval layer combines:

1. paper chunk semantic search,
2. claim semantic search,
3. graph-based paper expansion,
4. cross-encoder reranking.

This creates a stronger candidate relevance ranking than using plain keyword matching alone.

3.7 NLI contradiction analysis

The app tests relationships between claims using a CrossEncoder NLI model, with labels such as:

- SUPPORTS
- CONTRADICTS
- NEUTRAL

It then aggregates the relationships into:

- supporting pair count,
- contradicting pair count,
- neutral pair count,
- consensus level,
- contradiction summary.

This helps answer not only “what is the literature saying?” but also “where are the disputes?”

3.8 Evidence synthesis and answer generation

The app synthesizes a final answer using retrieved evidence and contradiction analysis. The result is generated in a structured format with sections such as:

- Scientific Summary
- Evidence Analysis
- Conflicting Evidence
- Conclusion and Implications

The answer is grounded in the retrieved paper list and citation references like [1], [2], and so on.

3.9 Interactive web UI

The Streamlit UI offers:

- query box for biomedical questions,
- paper count controls,
- toggle for LLM use,
- toggle for reranking,
- quick example prompts,
- live pipeline status updates,
- evidence summary cards,
- contradiction panels,
- paper-by-paper evidence review,
- filters by RCT / meta-analysis / other study types.

3.10 Local fallback mode

If Ollama is not available, the system still works in a fast heuristic mode with:

- rule-based extraction,
- fast semantic polarity logic,
- fallback synthesis patterns.

This keeps the experience usable even without a running local LLM.

4. Technology stack used

4.1 Application layer

- Python
- Streamlit
- dotenv for environment configuration

4.2 Data acquisition and web access

- requests
- httpx
- Europe PMC REST API

4.3 Document parsing

- BeautifulSoup
- lxml

4.4 LLM and local inference

- Ollama
- Qwen3:8b (default model)
- local model deployment for extraction and synthesis

4.5 Embeddings and similarity search

- SentenceTransformers
- all-MiniLM-L6-v2

4.6 Vector database

- ChromaDB

4.7 Knowledge graph

- NetworkX

4.8 Reranking and NLI

- CrossEncoder from sentence-transformers
- ms-marco-MiniLM-L-6-v2 for reranking
- nli-deberta-v3-base for contradiction / entailment classification

4.9 Scientific processing and utilities

- numpy
- pandas
- pydantic
- tenacity
- tqdm

4.10 Visualization / UI polish

- Plotly
- custom CSS in Streamlit

5. What makes the app different from basic RAG systems

A traditional RAG chatbot may do one of the following:

- retrieve a few documents,
- summarize them,
- answer a question.

This app does more than that:

- it extracts atomic research claims from each paper,
- structures them into PICO and evidence metadata,
- compares claims across paper boundaries,
- identifies contradictory findings,
- ranks evidence via both semantic and graph structure,
- produces a more review-like synthesis.

In short, it tries to behave more like a biomedical evidence review assistant than an ordinary document chatbot.

6. Similar technologies and how this project compares

6.1 Europe PMC / PubMed / PubMed Central

These are the closest direct literature search systems for biomedical research.

What they do well:

- access large literature databases
- index titles, metadata, and abstracts
- provide citation and publication discovery

What they do not do well:

- deep synthesis across multiple papers
- contradiction analysis
- full evidence review in one interface
- claim-level extraction and structured comparison

Our app extends this by adding scientific synthesis and claim comparison on top of the literature search.

6.2 Elicit

Elicit helps researchers find and synthesize evidence from papers.

Similarities:

- literature discovery
- evidence extraction
- synthesized answers

Differences:

- this project is local-first and open-source in spirit,
- it uses a graph + vector hybrid retrieval system,
- it includes contradiction detection across claims,
- it is built around an integrated local biomedical workflow rather than a SaaS-only interface.

6.3 Consensus / Semantic Scholar / Connected Papers

These platforms are strong for literature discovery and citation exploration.

Similarities:

- paper discovery
- literature relevance ranking
- concept-based exploration

Differences:

- they are not usually designed to do full local evidence synthesis over a custom biomedical question,
- they do not usually combine knowledge graph expansion with claim-level contradiction analysis,
- the app here is designed to be more evidence-review oriented and more transparent about the reasoning pipeline.

6.4 PaperQA / research copilot systems

These systems are often more general RAG or question-answering systems over research papers.

Similarities:

- retrieval from papers
- answer generation
- scientific question answering

Differences:

- this project is more specialized toward biomedical literature synthesis,
- it explicitly models evidence-level and contradiction dimensions,
- it uses a local open-source toolchain rather than depending entirely on a remote proprietary pipeline.

7. What we have done and what we want to do differently

7.1 What we have already built

The current system already includes:

- live Europe PMC search
- full paper retrieval
- XML parsing of biomedical article structure
- structured scientific extraction (PICO + research metadata)
- atomic claim extraction
- ChromaDB vector indexing
- NetworkX graph construction
- hybrid retrieval with reranking
- contradiction analysis
- final answer synthesis in a Streamlit UI

- fallback heuristic mode when LLM is unavailable

7.2 What we want to improve

The next phase could include:

1. Stronger biomedical entity normalization

- map terms to MeSH / UMLS / ontologies
- unify synonyms and variants
- improve concept matching across studies

2. Better evidence quality scoring

- weigh RCTs, systematic reviews, meta-analyses differently
- include risk of bias / study quality metadata
- provide a more formal evidence grading layer

3. Citation provenance and traceability

- show exact supporting fragments per claim
- link every claim back to the underlying paper and source section
- provide transparent reasoning chains

4. Better graph reasoning

- connect treatment, disease, population, and outcome nodes more deeply
- support multi-hop biomedical concept traversal
- allow network-level visual exploration

5. More robust clinical framing

- differentiate between mechanistic, observational, intervention, and safety evidence
- surface generalization limits of each study

6. More sophisticated synthesis evaluation

- benchmark against domain-specific answer quality metrics
- compare model outputs with human expert summaries
- add a “confidence + uncertainty” box for the final answer

7. Deployment improvements

- add user accounts or notebooks for reproducible reviews
 - persist project sessions and literature corpora
 - support larger literature corpora and batch review workflows
-

8. Detailed end-to-end pipeline

The current app follows a structured pipeline, which is one of its strongest points.

Step 1: User query input

The user enters a biomedical question in a Streamlit interface, such as:

- “Does acupuncture reduce pain in knee osteoarthritis?”
- “Is metformin effective for weight loss in non-diabetic obesity?”

Step 2: Search literature

The app calls the Europe PMC REST API using a query string plus open-access filtering.

It retrieves candidate papers with metadata such as title, abstract, authors, journal, DOI, and year.

Step 3: Download full-text XML

For each candidate paper, the app tries to fetch full-text XML from PMC. If the full text is unavailable, it falls back to the abstract only.

Step 4: Parse the document structure

The XML parser extracts structured sections and cleans raw text.

It identifies:

- abstract
- introduction
- methods
- results
- discussion
- conclusion
- key phrases and keywords

This cleaned document structure becomes the base evidence text for extraction.

Step 5: Claim extraction

The extraction module processes each paper and identifies key scientific information.

It can do one of two things:

- use Ollama + Qwen3 for LLM-driven structured extraction,
- or use a heuristic extraction layer if the model is unavailable.

The extracted output includes:

- PICO fields,
- methodology,
- effect-size hints,
- key findings,
- limitations,
- atomic claims.

Step 6: Index evidence in vector database

The app stores paper chunks and atomic claims into ChromaDB collections.

This creates searchable semantic representations of both:

- broader paper text,
- precise claim-level evidence.

Step 7: Build the knowledge graph

The graph store creates a NetworkX graph of the retrieved literature.

It links:

- papers to authors,
- papers to concepts,
- papers to claims,
- claim-to-claim support or contradiction edges.

This supports relationship-aware retrieval and multi-paper expansion.

Step 8: Hybrid retrieval and reranking

The app runs a hybrid retrieval process:

1. semantic search over paper chunks,
2. semantic search over claim embeddings,
3. graph expansion from top seed papers,
4. reranking using a cross-encoder.

This produces a ranked list of the most relevant papers and claims.

Step 9: Contradiction analysis

The system assembles the top claims across the retrieved papers and compares them pairwise using the DeBERTa-based NLI model.

For each pair it determines whether the claims:

- support one another,
- contradict one another,
- are neutral.

It aggregates this into a balance of evidence.

Step 10: Final evidence synthesis

The synthesis module compiles the selected papers, their relevant claims, and the contradiction analysis into a final answer.

The output is structured with markdown sections and explicit paper references such as [1], [2].

Step 11: UI rendering

The Streamlit app renders:

- scientific summary,
- evidence metrics,
- contradictory findings,
- paper cards with methodology badges,
- PICO metadata,
- key claims,
- relevant links and abstracts.

9. Summary of product value

This project sits between a literature search system and a biomedical evidence review assistant. It is useful for:

- research exploration,
- evidence synthesis,
- literature comparison,
- identifying conflicting findings,
- understanding what a study population and intervention were,
- summarizing multiple studies quickly.

The overall value is not just “find papers,” but “turn literature into structured evidence and compare it intelligently.”

10. One-line product description

A local-first biomedical evidence synthesis system that searches Europe PMC, extracts structured scientific claims, indexes them in vector and graph form, detects contradictory findings, and summarizes the literature into a research-grade answer.

